NLP Course
Search documentation Ctrl+K
EN | 2,358
0. SETUP
1. TRANSFORMER MODELS
2. USING 🤗 TRANSFORMERS
3. FINE-TUNING A PRETRAINED MODEL
Introduction
Processing the data
Fine-tuning a model with the Trainer API or Keras
A full training
Fine-tuning, Check!
End-of-chapter quiz
4. SHARING MODELS AND TOKENIZERS
5. THE 🤗 DATASETS LIBRARY
6. THE 🤗 TOKENIZERS LIBRARY
7. MAIN NLP TASKS
8. HOW TO ASK FOR HELP
9. BUILDING AND SHARING DEMOS NEW
COURSE EVENTS

Join the Hugging Face community
and get access to the augmented documentation experience
Collaborate on models, datasets and Spaces
Faster examples with accelerated inference
Switch between documentation themes
Sign Up to get started

Pytorch | TensorFlow | Ask a question | Open in Colab | Open In Studio Lab

Processing the data
Loading a dataset from the Hub
Preprocessing a dataset
Dynamic padding

Processing the data

Continuing with the example from the previous chapter, here is how we would train a sequence classifier on one batch in PyTorch:

```
import torch
from transformers import AdamW, AutoTokenizer, AutoModelForSequenceClassification

# Same as before
checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
sequences = [
    "I've been waiting for a HuggingFace course my whole life.",
    "This course is amazing!",
]
batch = tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")

# This is new
batch["labels"] = torch.tensor([1, 1])

optimizer = AdamW(model.parameters())
loss = model(**batch).loss
loss.backward()
optimizer.step()
```

Of course, just training the model on two sentences is not going to yield very good results. To get better results, you will need to prepare a bigger dataset.

In this section we will use as an example the MRPC (Microsoft Research Paraphrase Corpus) dataset, introduced in a paper by William B. Dolan and Chris Brockett. The dataset consists of 5,801 pairs of sentences, with a label indicating if they are paraphrases or not (i.e., if both sentences mean the same thing). We've selected it for this chapter because it's a small dataset, so it's easy to experiment with training on it.

Loading a dataset from the Hub

The Hub doesn't just contain models; it also has multiple datasets in lots of different languages. You can browse the datasets here, and we recommend you try to load and process a new dataset once you have gone through this section (see the general documentation here). But for now, let's focus on the MRPC dataset! This is one of the 10 datasets composing the GLUE benchmark, which is an academic benchmark that is used to measure the performance of ML models across 10 different text classification tasks.

The 🤗 Datasets library provides a very simple command to download and cache a dataset on the Hub. We can download the MRPC dataset like this:

```
from datasets import load_dataset

raw_datasets = load_dataset("glue", "mrpc")
raw_datasets
```

```
DatasetDict({
    train: Dataset({
        features: ['sentence1', 'sentence2', 'label', 'idx'],
        num_rows: 3668
    })
    validation: Dataset({
        features: ['sentence1', 'sentence2', 'label', 'idx'],
        num_rows: 408
    })
    test: Dataset({
        features: ['sentence1', 'sentence2', 'label', 'idx'],
        num_rows: 1725
    })
})
```

As you can see, we get a `DatasetDict` object which contains the training set, the validation set, and the test set. Each of those contains several columns (`sentence1`, `sentence2`, `label`, and `idx`) and a variable number of rows, which are the number of elements in each set (so, there are 3,668 pairs of sentences in the training set, 408 in the validation set, and 1,725 in the test set).

This command downloads and caches the dataset, by default in *~/.cache/huggingface/datasets*. Recall from Chapter 2 that you can customize your cache folder by setting the `HF_HOME` environment variable.

We can access each pair of sentences in our `raw_datasets` object by indexing, like with a dictionary:

```
raw_train_dataset = raw_datasets["train"]
raw_train_dataset[0]
```

```
{'idx': 0,
 'label': 1,
 'sentence1': 'Amrozi accused his brother , whom he called " the witness " , of deliberately distorting h
 'sentence2': 'Referring to him as only " the witness " , Amrozi accused his brother of deliberately dist
```

We can see the labels are already integers, so we won't have to do any preprocessing there. To know which integer corresponds to which label, we can inspect the `features` of our `raw_train_dataset`. This will tell us the type of each column:

```
raw_train_dataset.features
```

```
{'sentence1': Value(dtype='string', id=None),
 'sentence2': Value(dtype='string', id=None),
 'label': ClassLabel(num_classes=2, names=['not_equivalent', 'equivalent'], names_file=None, id=None),
 'idx': Value(dtype='int32', id=None)}
```

Behind the scenes, `label` is of type `ClassLabel`, and the mapping of integers to label name is stored in the *names* folder. `0` corresponds to `not_equivalent`, and `1` corresponds to `equivalent`.

> ✏️ **Try it out!** Look at element 15 of the training set and element 87 of the validation set. What are their labels?

Preprocessing a dataset

To preprocess the dataset, we need to convert the text to numbers the model can make sense of. As you saw in the previous chapter, this is done with a tokenizer. We can feed the tokenizer one sentence or a list of sentences, so we can directly tokenize all the first sentences and all the second sentences of each pair like this:

```
from transformers import AutoTokenizer

checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
tokenized_sentences_1 = tokenizer(raw_datasets["train"]["sentence1"])
tokenized_sentences_2 = tokenizer(raw_datasets["train"]["sentence2"])
```

However, we can't just pass two sequences to the model and get a prediction of whether the two sentences are paraphrases or not. We need to handle the two sequences as a pair, and apply the appropriate preprocessing. Fortunately, the tokenizer can also take a pair of sequences and prepare it the way our BERT model expects:

```
inputs = tokenizer("This is the first sentence.", "This is the second one.")
inputs
```

```
{
  'input_ids': [101, 2023, 2003, 1996, 2034, 6251, 1012, 102, 2023, 2003, 1996, 2117, 2028, 1012, 102],
  'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
  'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
}
```

We discussed the `input_ids` and `attention_mask` keys in Chapter 2, but we put off talking about `token_type_ids`. In this example, this is what tells the model which part of the input is the first sentence and which is the second sentence.

> ✏️ **Try it out!** Take element 15 of the training set and tokenize the two sentences separately and as a pair. What's the difference between the two results?

If we decode the IDs inside `input_ids` back to words:

```
tokenizer.convert_ids_to_tokens(inputs["input_ids"])
```

we will get:

```
['[CLS]', 'this', 'is', 'the', 'first', 'sentence', '.', '[SEP]', 'this', 'is', 'the', 'second', 'one', '
```

So we see the model expects the inputs to be of the form `[CLS] sentence1 [SEP] sentence2 [SEP]` when there are two sentences. Aligning this with the `token_type_ids` gives us:

```
['[CLS]', 'this', 'is', 'the', 'first', 'sentence', '.', '[SEP]', 'this', 'is', 'the', 'second', 'one', '
[      0,      0,    0,     0,       0,          0,   0,       0,      1,    1,     1,        1,     1,
```

As you can see, the parts of the input corresponding to `[CLS] sentence1 [SEP]` all have a token type ID of `0`, while the other parts, corresponding to `sentence2 [SEP]`, all have a token type ID of `1`.

Note that if you select a different checkpoint, you won't necessarily have the `token_type_ids` in your tokenized inputs (for instance, they're not returned if you use a DistilBERT model). They are only returned when the model will know what to do with them, because it has seen them during its pretraining.

Here, BERT is pretrained with token type IDs, and on top of the masked language modeling objective we talked about in Chapter 1, it has an additional objective called *next sentence prediction*. The goal with this task is to model the relationship between pairs of sentences.

With next sentence prediction, the model is provided pairs of sentences (with randomly masked tokens) and asked to predict whether the second sentence follows the first. To make the task non-trivial, half of the time the sentences follow each other in the original document they were extracted from, and the other half of the time the two sentences come from two different documents.

In general, you don't need to worry about whether or not there are `token_type_ids` in your tokenized inputs: as long as you use the same checkpoint for the tokenizer and the model, everything will be fine as the tokenizer knows what to provide to its model.

Now that we have seen how our tokenizer can deal with one pair of sentences, we can use it to tokenize our whole dataset: like in the previous chapter, we can feed the tokenizer a list of pairs of sentences by giving it the list of first sentences, then the list of second sentences. This is also compatible with the padding and truncation options we saw in Chapter 2. So, one way to preprocess the training dataset is:

```
tokenized_dataset = tokenizer(
    raw_datasets["train"]["sentence1"],
    raw_datasets["train"]["sentence2"],
    padding=True,
    truncation=True,
)
```

This works well, but it has the disadvantage of returning a dictionary (with our keys, `input_ids`, `attention_mask`, and `token_type_ids`, and values that are lists of lists). It will also only work if you have enough RAM to store your whole dataset during the tokenization (whereas the datasets from the 🤗 Datasets library are Apache Arrow files stored on the disk, so you only keep the samples you ask for loaded in memory).

To keep the data as a dataset, we will use the `Dataset.map()` method. This also allows us some extra flexibility, if we need more preprocessing done than just tokenization. The `map()` method works by applying a function on each element of the dataset, so let's define a function that tokenizes our inputs:

```
def tokenize_function(example):
    return tokenizer(example["sentence1"], example["sentence2"], truncation=True)
```

This function takes a dictionary (like the items of our dataset) and returns a new dictionary with the keys `input_ids`, `attention_mask`, and `token_type_ids`. Note that it also works if the `example` dictionary contains several samples (each key as a list of sentences) since the `tokenizer` works on lists of pairs of sentences, as seen before. This will allow us to use the option `batched=True` in our call to `map()`, which will greatly speed up the tokenization. The `tokenizer` is backed by a tokenizer written in Rust from the 🤗 Tokenizers library. This tokenizer can be very fast, but only if we give it lots of inputs at once.

Note that we've left the `padding` argument out in our tokenization function for now. This is because padding all the samples to the maximum length is not efficient: it's better to pad the samples when we're building a batch, as then we only need to pad to the maximum length in that batch, and not the maximum length in the entire dataset. This can save a lot of time and processing power when the inputs have very variable lengths!

Here is how we apply the tokenization function on all our datasets at once. We're using `batched=True` in our call to `map` so the function is applied to multiple elements of our dataset at once, and not on each element separately. This allows for faster preprocessing.

```
tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
tokenized_datasets
```

The way the 🤗 Datasets library applies this processing is by adding new fields to the datasets, one for each key in the dictionary returned by the preprocessing function:

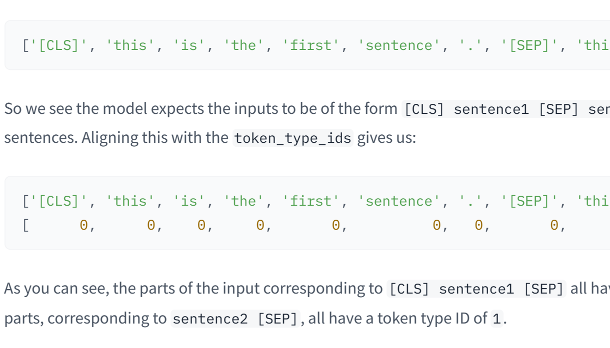
```
DatasetDict({
    train: Dataset({
        features: ['attention_mask', 'idx', 'input_ids', 'label', 'sentence1', 'sentence2', 'token_type_i
        num_rows: 3668
    })
    validation: Dataset({
        features: ['attention_mask', 'idx', 'input_ids', 'label', 'sentence1', 'sentence2', 'token_type_i
        num_rows: 408
    })
    test: Dataset({
        features: ['attention_mask', 'idx', 'input_ids', 'label', 'sentence1', 'sentence2', 'token_type_i
        num_rows: 1725
    })
})
```

You can even use multiprocessing when applying your preprocessing function with `map()` by passing along a `num_proc` argument. We didn't do this here because the 🤗 Tokenizers library already uses multiple threads to tokenize our samples faster, but if you are not using a fast tokenizer backed by this library, this could speed up your preprocessing.

Our `tokenize_function` returns a dictionary with the keys `input_ids`, `attention_mask`, and `token_type_ids`, so those three fields are added to all splits of our dataset. Note that we could also have changed existing fields if our preprocessing function returned a new value for an existing key in the dataset to which we applied `map()`.

The last thing we will need to do is pad all the examples to the length of the longest element when we batch elements together — a technique we refer to as *dynamic padding*.

Dynamic padding



The function that is responsible for putting together samples inside a batch is called a *collate function*. It's an argument you can pass when you build a `DataLoader`, the default being a function that will just convert your samples to PyTorch tensors and concatenate them (recursively if your elements are lists, tuples, or dictionaries). This won't be possible in our case since the inputs we have won't all be of the same size. We have deliberately postponed the padding, to only apply it as necessary on each batch and avoid having over-long inputs with a lot of padding. This will speed up training by quite a bit, but note that if you're training on a TPU it can cause problems — TPUs prefer fixed shapes, even when that requires extra padding.

To do this in practice, we have to define a collate function that will apply the correct amount of padding to the items of the dataset we want to batch together. Fortunately, the 🤗 Transformers library provides us with such a function via `DataCollatorWithPadding`. It takes a tokenizer when you instantiate it (to know which padding token to use, and whether the model expects padding to be on the left or on the right of the inputs) and will do everything you need:

```
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

To test this new toy, let's grab a few samples from our training set that we would like to batch together. Here, we remove the columns `idx`, `sentence1`, and `sentence2` as they won't be needed and contain strings (and we can't create tensors with strings) and have a look at the lengths of each entry in the batch:

```
samples = tokenized_datasets["train"][:8]
samples = {k: v for k, v in samples.items() if k not in ["idx", "sentence1", "sentence2"]}
[len(x) for x in samples["input_ids"]]
```

```
[50, 59, 47, 67, 59, 50, 62, 32]
```

No surprise, we get samples of varying length, from 32 to 67. Dynamic padding means the samples in this batch should all be padded to a length of 67, the maximum length inside the batch. Without dynamic padding, all of the samples would have to be padded to the maximum length in the whole dataset, or the maximum length the model can accept. Let's double-check that our `data_collator` is dynamically padding the batch properly:

```
batch = data_collator(samples)
{k: v.shape for k, v in batch.items()}
```

```
{'attention_mask': torch.Size([8, 67]),
 'input_ids': torch.Size([8, 67]),
 'token_type_ids': torch.Size([8, 67]),
 'labels': torch.Size([8])}
```

Looking good! Now that we've gone from raw text to batches our model can deal with, we're ready to fine-tune it!

> ✏️ **Try it out!** Replicate the preprocessing on the GLUE SST-2 dataset. It's a little bit different since it's composed of single sentences instead of pairs, but the rest of what we did should look the same. For a harder challenge, try to write a preprocessing function that works on any of the GLUE tasks.